

Kite Algo Trader — Detailed Prep

In This Edition

1	0. How to use this in an interview	2
2	1. The pitch (30--45 seconds)	2
3	2. Technologies & tools (and the <i>why</i> behind each)	2
4	3. Architecture in six boxes	3
5	4. Problem-solving stories (STAR) --- the core of the interview	4
	5.1 (1) Duplicate orders from a thread-lifecycle race flagship story	4
	5.2 (2) Watchdog killing <i>healthy</i> strategies (liveness $\neq$ progress)	4
	5.3 (3) Expired options stranded forever (never gate an exit on data that can vanish) . . .	5
	5.4 (4) A ₹33k phantom loss from a fabricated price	5
	5.5 (5) The 1 GB VM core-dumping with SIGBUS	5
	5.6 (6) NSE 403-blocking cloud IPs (don't trust a single vendor)	6
	5.7 (7) API rate-limit storms	6
	5.8 (8) eventlet broke the SSL stack (async monkey-patching is leaky)	6
	5.9 (9) Crash recovery & state drift	6
	5.10 (10) A refactor silently broke DB reads (my own bug)	6
	5.11 ■ Backtests that didn't match live	7
6	5. STAR answers to the classic interview prompts	7
7	6. Top 20 likely interview questions --- with answers	9
8	7. Top 10 deep-dive technical questions --- with answers	12
9	8. Top 10 follow-up questions (experienced interviewer) --- with answers	14
10	9. Where the interviewer may challenge you --- and how to defend	17
11	10. Things to study/refresh before the interview	17
12	11. One-line closer	18

Personal notes. **Untracked on purpose — do not commit.** Companion to INTERVIEW_PREP.md (the quick-reference). This one is the long-form, STAR-format, "talk for 5–10 minutes" version. Read the pitch + 3 stories cold; skim the rest.

1 0. How to use this in an interview

- › **Lead with reliability, not strategies.** The senior-level signal is "I kept a stateful, real-time, money-handling system alive 24/7," not "I wrote trading logic."
- › **Tell stories, not features.** Every strong answer below is a real incident with a symptom → investigation → root cause → fix → lesson arc.
- › **Always name the trade-off.** Interviewers reward "I chose X knowing it cost me Y, because Z mattered more for a solo free-tier system."
- › **Own the mistakes.** Two bugs were caused by *my own refactors* (path-depth, fabricated fill). Volunteering those reads as maturity, not weakness.

2 1. The pitch (30--45 seconds)

"It's a multi-strategy algorithmic trading platform for Indian markets — equities, index options, and MCX commodities. Around 31 strategies run concurrently, in paper or live mode through Zerodha. It ingests real-time data over WebSockets, runs its own paper/live order engine, has a full observability stack — Grafana, Loki, metrics, error tracking — and a backtesting engine that replays the *actual* strategy code rather than a reimplementation. It runs 24/7 on two cloud VMs.

The interesting engineering wasn't the strategies — those are the easy part. It was making a stateful, real-time, money-handling system *reliable*: surviving crashes without losing open positions, surviving feed outages, not double-trading under race conditions, and running inside a 1 GB free-tier VM without getting OOM-killed."

3 2. Technologies & tools (and the *why* behind each)

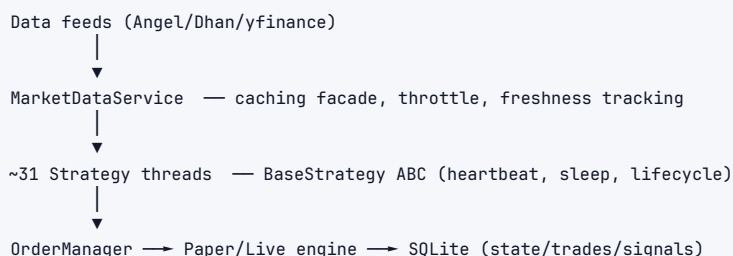
Layer	Tech	Why this, not the alternative
Language	Python 3	Quant ecosystem (numpy/pandas), broker SDKs are Python-first, fast iteration for a solo dev
Web/Ba	Flask + Flask-SocketIO + eventlet	Needed server→browser push (live ticks, P&L). Eventlet green threads give async I/O in one process — no separate async runtime to operate
Realtime feeds	Angel One SmartAPI (WS+REST, primary), Dhan (WS option ticks), yfinance (fallback)	Multi-vendor behind one facade; no single point of failure for data
Executio	KiteConnect/Zerodha (live + OAuth), custom PaperEngine	Paper-first by default; live is opt-in per strategy
OLTP stor- age	SQLite (WAL mode)	Live state, trades, signals. Zero-ops, single-file, good enough for one writer; WAL gives concurrent reads

Layer	Tech	Why this, not the alternative
OLAP storage	DuckDB + Parquet (hot/cold tiering)	Columnar analytics + backtest data without standing up a DB server; cold tier is just Parquet files
Frontend	Vanilla JS SPA + Socket.IO + PWA/service worker + Lightweight Charts	Mobile-installable, offline shell, no framework/build overhead to maintain solo
Quant	Pure-Python indicators (EMA/ATR/RSI/ADX/SuperTrend/VWAP), Black–Scholes greeks, QuantLib (IV), pandera (data validation), Optuna (optimization)	Transparent, debuggable math; Optuna for walk-forward param search
Observability	Prometheus, Grafana, Loki , process-exporter/Alloy, GlitchTip (Sentry-compatible)	RED metrics + logs + error tracking across both hosts; GlitchTip is self-hosted so it's free
Infra	Oracle Cloud (1 GB micro + 24 GB ARM), systemd units+timers, nginx (TLS), OCI Object Storage (DB backups w/ lifecycle), bash deploy	Free-tier-constrained; systemd is the supervisor + scheduler
CI/CD	GitHub Actions (ruff, pytest, gitleaks, pip-audit, Dependabot), one-command multi-host deploy script	Lint + test + secret-scan + dependency-audit on every push

Anticipated "why" questions and one-liners:

- › *Why SQLite not Postgres?* Solo, free-tier, one writer. Zero ops beat concurrency-safety — and I learned where that line is (the SIGBUS story).
- › *Why eventlet not asyncio/threads?* Socket.IO integration + one-process I/O concurrency. It bit me (SSL monkey-patching), but for the scale it was the right operational simplicity.
- › *Why DuckDB + Parquet not just SQLite?* Backtests scan millions of rows columnar; SQLite row-store would be slow and bloated. DuckDB reads Parquet directly, no import step.
- › *Why two VMs of different sizes?* Free tier. The 1 GB micro forced real resource discipline (caps, tiering) — same code runs on the 24 GB ARM box by changing env vars.

4 3. Architecture in six boxes



Three cross-cutting **safety systems**:

- › **Watchdog** — detects strategies whose heartbeat is stale and recycles them.
- › **TradeResumer** — on boot, restores open trades from SQLite so a restart doesn't go blind.
- › **Reconciler** — continuously detects position↔state drift (a held position with no tracking state).

Plus a **backtest engine** that replays the real strategy code, and the **observability stack**.

5 4. Problem-solving stories (STAR) --- the core of the interview

These are ordered by how impressive/teachable they are. Know stories (1)(2)(3) verbatim; the rest are "I can also tell you about..." material.

5.1 (1) Duplicate orders from a thread-lifecycle race flagship story

Situation. I noticed in the trade journal that one strategy had bought the *same* option four times within about a minute. Positions were accumulating, and sometimes legs were left open after a "close," which meant silent, real losses.

Task. Figure out how a single strategy could fire the same entry multiple times — and kill the whole class of bug, not just patch the one symptom.

Action. I traced it back through the logs to the watchdog. The watchdog's job is to recycle a "stuck" strategy with `stop(); start()`. But `stop()` only flipped a boolean flag — it never *joined* the thread. So if a strategy looked stuck because it was hung on a blocking call, the old thread was still alive when `start()` ran, and `start()` happily spawned a *second* thread. Now two threads were running the same strategy, each independently hitting the entry condition and placing orders. The "fix it later" version would've been to dedupe orders downstream — but that just hides a process running twice. So I fixed the lifecycle: `start()` now refuses to spawn while the previous thread is still alive, and the recycle became two-phase — stop, drain, *then* restart — so there's never a window with two live threads. I also added a 15:25 EOD square-off net to clean up anything that slipped through.

Result. The duplicate-entry class disappeared. The mental model I took away: **a kill that doesn't join is not a kill**. Any supervisor that restarts work has to guarantee the old worker is actually dead first, otherwise "restart" becomes "duplicate."

Lesson / one-liner. "Thread lifecycle and idempotency: a restart primitive is only safe if it's also a *join*."

5.2 (2) Watchdog killing healthy strategies (liveness ≠ progress)

Situation. After adding the watchdog, strategies started getting recycled constantly — every few minutes early on, and later a stubborn daily recycle around 14:54 for one specific strategy.

Task. Stop the watchdog from euthanizing strategies that were actually fine, without making it blind to genuine hangs.

Action. The watchdog recycles anything whose heartbeat is stale. The problem was that idle and wait loops used a plain `time.sleep()` — during a long sleep, no heartbeat went out, so a perfectly healthy strategy *looked* hung. The naive fix is "make the timeout longer," but that just makes you slower to catch real hangs. Instead I introduced a heartbeat-aware `self._sleep()`: it sleeps in short interruptible chunks and emits a heartbeat roughly every 30 seconds. So a strategy that's deliberately idle keeps signaling "alive," but a strategy that's genuinely wedged *outside* the sleep (stuck on a blocking call) still goes stale and gets caught. The daily 14:54 recycle was the last straggler — a post-force-exit wait loop that still used raw `time.sleep(60)`. I swept the codebase and converted ~74 raw `time.sleep()` calls to the heartbeat-aware sleep.

Result. Needless recycles went away; real hangs are still detected. The conceptual win is the distinction between **"the process is alive"** and **"the work is making progress"** — the liveness signal has to be emitted *by the work loop itself*, not inferred from the process being up.

Interview gold: there's a moment here where the interviewer (playing me) had it backwards — I initially assumed `time.sleep` was *deliberately* used to avoid recycling. I checked git history and found the opposite: the heartbeat sleep was added *on purpose* to stop needless recycles, and the remaining `time.sleep` calls were just un-migrated leftovers. **Verify-**

ing a plausible assumption against the actual history instead of trusting memory is a good story on its own.

5.3 (3) Expired options stranded forever (never gate an exit on data that can vanish)

Situation. An Iron Condor opened on the 16th was still showing open legs on the 17th, even though the options had expired. And this had *recurred* — we'd "fixed" an expiry strand before and it came back.

Task. Get expired positions to actually settle, and find why the earlier fix didn't hold.

Action. There were three compounding root causes:

1. **The exit was gated behind a live price.** The close logic computed net premium first, and if it couldn't (`net < 0 / missing`), it retried. But expired options *stop ticking* — the price is gone forever — so it retried until expiry and never exited.
2. **The expiry safety-net was itself gated** behind `if self._in_trade`, which was `False` whenever the in-memory trade state had been lost (e.g. after a restart) — exactly the case where you most need the net.
3. **The paper engine rejected ₹0 settlements** (a `price ≤ 0` guard), so even when the net *did* fire, the settlement order bounced.

I fixed all three: a **state-independent sweep** that reads positions directly, parses the expiry out of the option symbol, and settles expired legs at **intrinsic-vs-spot** with no LTP needed; the paper engine now accepts a ₹0 fill *only* for a genuine expired-option close; and a re-entry guard so a strategy won't open a fresh position group while it still holds untracked legs.

Result. Expired strands cleared on both VMs and stopped recurring. The durable lesson: **never gate a critical exit behind data that can disappear.** Expiry is a *time* fact — derive it from the clock and the symbol, not from a price feed that goes silent precisely when you need to act.

5.4 (4) A ₹33k phantom loss from a fabricated price

Situation. A close booked a large loss that didn't match reality.

Task. Find where a wrong number entered the money path.

Action. When the live price was missing, the close had a hard-coded `or 0.5` fallback — it *fabricated* a fill price to keep going. That fabricated number flowed straight into P&L.

Result / lesson. Removed all fabricated fallbacks: retry, then skip, then settle at intrinsic or not at all. **In a money system, a wrong number is more dangerous than no number** — "no data" has to be a first-class, handled state, never papered over with a default.

5.5 (5) The 1 GB VM core-dumping with SIGBUS

Situation. The micro VM kept crashing the whole trading process with SIGBUS (a bus error / core dump), seemingly at random.

Task. Stop the crashes on a box I couldn't make bigger (free tier).

Action. SIGBUS pointed at memory-mapped I/O. DuckDB defaults to using ~80% of RAM; running concurrently with the trader on a 1 GB box, that over-committed memory. Under pressure the kernel swap-thrashed, and a page fault on an **mmap'd Parquet page** that couldn't be backed surfaced as SIGBUS — which kills the process, not just the query. I capped DuckDB's memory and thread count and gave it a disk-spill temp dir (all env-overridable so the big VM can use more). I also disabled SQLite's mmap, which had the same SIGBUS-under-concurrent-write failure mode. `systemd Restart=on-failure` is the backstop so any residual crash self-heals.

Result / lesson. Crashes stopped. The takeaway is understanding *why mmap faults become SIGBUS* and that **defaults tuned for a workstation are dangerous in a constrained container** — resource limits aren't optional on small hosts.

5.6 (6) NSE 403-blocking cloud IPs (don't trust a single vendor)

Situation. A data feed worked perfectly on my laptop but silently went stale on the VMs.

Task. Find why prod-only, and de-risk the dependency.

Action. NSE blocks data-center IP ranges with 403s. Locally I had a residential IP; in the cloud I was blocked — and the failure was *silent staleness*, not an error. Because all data already flowed through the MarketDataService facade, I migrated every live feed (including India VIX) to Angel One and gated the dead NSE path behind a flag. The 31 strategies didn't change a line.

Result / lesson. The facade paid for itself — **abstract external vendors behind your own interface** so swapping one is a one-module change, and **treat "works locally" as unproven in prod** when third-party access is environmental.

5.7 (7) API rate-limit storms

Situation. Angel One started throttling — roughly 1,400 hits/host/day — and candle fetches came back empty, leaving strategies blind.

Task. Get under the limit without losing data freshness.

Action. The standard three-part pattern: a **cache** so N strategies asking for the same candle = one fetch; a **process-wide throttle** (~3 req/s) so bursts can't stampede; and **exponential backoff** (5→10→20→40s) on throttle responses, with alerting. I captured a pre-fix baseline from metrics so I could *prove* the improvement.

Result / lesson. ~90% fewer API hits, strategies stayed fed. **Cache + throttle + backoff is the canonical rate-limit triad**, and baselining before/after with metrics is how you show it worked.

5.8 (8) eventlet broke the SSL stack (async monkey-patching is leaky)

Situation. After enabling eventlet, Telegram notifications and yfinance calls hit infinite recursion.

Task. Keep cooperative concurrency without breaking blocking HTTP libraries.

Action. eventlet monkey-patches the standard SSL/socket stack to make it cooperative; some libraries don't tolerate the patched stack and recurse. The fix was to fetch the *original*, un-patched `urllib.request` via `eventlet.patcher.original()` for those specific calls, so they use real blocking sockets while everything else stays green.

Result / lesson. **Know your concurrency model's failure modes.** Monkey-patching the runtime is convenient but leaks into every library that touches sockets — you need an escape hatch.

5.9 (9) Crash recovery & state drift

Situation. A restart could drop in-memory trade state while real positions stayed open — the strategy would come back blind, re-enter, and accumulate orphans.

Task. Make restarts safe for a stateful, money-handling process.

Action. Three layers: **TradeResumer** restores open trades from SQLite on boot (resume, not re-fire); the **Reconciler** continuously flags drift — a position with no matching trade state — and exposes it as a metric; and a re-entry guard refuses a new position group while untracked legs are held.

Result / lesson. Restarts became boring. **Stateful systems need explicit crash recovery plus a reconciliation loop** — you can't assume in-memory state and the broker's reality stay in sync across a restart.

5.10 (10) A refactor silently broke DB reads (my own bug)

Situation. After I reorganized the code into deeper packages, the History and P&L tabs went blank — but the app booted fine and tests passed.

Task. Find why reads returned nothing with no error anywhere.

Action. The DB path was computed from `__file__` with a fixed number of `os.path.dirname()` calls. Moving the module one directory deeper changed the relative depth, so the path resolved to a *different, empty* file. Nothing errored: an empty SQLite DB just returns zero rows, and the tests had been *overriding* the path so they never exercised the real resolution. I corrected the path math and added regression tests that pin every `__file__`-derived path to the repo root.

Result / lesson. "It imports and boots" $\neq$ "it works," and **depth-changing refactors are sneaky** because relative-path math breaks silently. Tests that mock the thing they should verify give false confidence.

5.11 ■ Backtests that didn't match live

Situation. The original backtester used per-strategy *adapters* — reimplementations of the strategy logic for backtesting — and results drifted from live behavior.

Task. Make backtests trustworthy.

Action. I rebuilt the engine to replay the **real strategy code** against recorded market data — same class, same logic, no adapter. I added a **data-fidelity grade** so you know how trustworthy a given backtest's inputs were, Optuna optimization with walk-forward validation, and ran it as a niced subprocess so a heavy backtest never starves the live trader.

Result / lesson. Backtest and live can't diverge if they're literally the same code. **If your backtest runs different code than production, the backtest is lying to you** — parity is the whole game.

6 5. STAR answers to the classic interview prompts

Conversational, ~1–2 minutes each. Pick the matching story above and compress.

"Why did you choose this technology?" (SQLite/eventlet)

(S) Solo project, free-tier cloud, real money. (T) I needed something I could operate alone with near-zero ops. (A) I chose SQLite over Postgres and eventlet over a heavier async stack — single-process, single-file, no servers to babysit. (R) It let me move fast and the system ran 24/7. (Trade-off I name out loud) both choices later bit me — SQLite's mmap caused SIGBUS on the 1 GB box, eventlet's monkey-patching broke SSL libs — and I can tell you exactly where I'd switch to Postgres + a real task runner if this grew past one operator.

"What was the hardest technical challenge?"

→ Story (1) (duplicate orders). Emphasize: symptom in the journal → traced to the watchdog → `stop()` never joined → spawn guard + two-phase recycle. "Hardest because the bug only appeared when a strategy was *already* in a bad state, so it was rare and timing-dependent — and the tempting fix (dedupe orders) would've hidden a process running twice instead of fixing it."

"Tell me about a production issue you faced."

→ Story (3) (expired-option strand) or (5) (SIGBUS). Both are clean "real money / real crash" incidents with a layered root cause.

"Describe a scaling / resource problem."

→ Story (5) (1 GB SIGBUS) + (7) (rate-limit storm). Frame (5) as scaling *down*: "the interesting constraint wasn't traffic, it was running a real-time system in 1 GB — which forced memory caps and hot/cold data tiering that a bigger box would've let me get lazy about."

"Tell me about a design decision that backfired."

→ Story (10) (path-depth refactor) — *my own* bug — or the SQLite/eventlet choices. "The refactor made the code cleaner but broke `__file__`-relative paths silently. It backfired because it passed every check I had — it imported, it booted, tests were green — and still served an empty database. That taught me to distrust tests that mock the exact thing they claim to verify."

"What trade-offs did you make?"

Operational simplicity vs concurrency-safety (SQLite/single-process), freshness vs API budget (cache TTL vs rate limits), latency vs reliability (retry/skip instead of fabricating a fill), and feature velocity vs test coverage (I shipped fast, then back-filled tests after incidents — which I'd reverse if doing it again).

"How did you debug difficult issues?"

Almost everything started at a **dashboard or a log**, not in the code. RED metrics in Grafana, structured logs in Loki, errors in GlitchTip. The duplicate-order bug surfaced in the trade journal; the recycle bug in the watchdog logs; the feed outage as a *freshness* metric going stale rather than an exception. Then I reproduce, find the *root* cause not the symptom, and add the test that would've caught it.

"How did you ensure reliability and maintainability?"

Defense in depth: heartbeat + watchdog (detection), TradeResumer (recovery), Reconciler (drift detection), systemd Restart=on-failure (floor), and CI (ruff/pytest/gitleaks/pip-audit) on every push. Crucially, **every safety system was added in response to a real incident** — I didn't build them speculatively.

"If given more time, what would you improve?"

Move to Postgres (kills the SQLite concurrency/mmap class of bug), front-load tests instead of back-filling them, replace eventlet with a cleaner async or real task model, and add automated end-to-end paper-trading smoke tests in CI so a regression like the blank-DB one can't ship.

"What was your personal contribution?"

Solo project — I designed, built, and operate all of it: the strategy framework, data facade, paper/live engines, the three safety systems, the backtest engine, the observability stack, and the deploy pipeline. More importantly, I *ran* it in production and every reliability feature came from an incident I diagnosed and fixed myself.

7 6. Top 20 likely interview questions --- with answers

1. Give me the high-level architecture and the data flow from feed to order.

Data comes in from broker WebSockets and REST — Angel One as primary, Dhan for option ticks, yfinance as a fallback. Everything goes through one **MarketDataService** facade that caches, throttles, and tracks freshness, so no strategy ever talks to a vendor directly. About 31 strategy threads — all subclasses of a **BaseStrategy** ABC — read from that facade, make decisions, and emit signals to an **OrderManager**, which routes to either the **PaperEngine** or the live Zerodha path. State, trades, and signals persist to **SQLite**. Wrapped around all of that are three safety systems — a **Watchdog**, a **TradeResumer**, and a **Reconciler** — plus an observability stack and a backtest engine. The one-line version: *feed → caching facade → strategy threads → order manager → paper/live engine → SQLite, with detection/recovery/drift loops cross-cutting all of it.*

2. Why SQLite instead of a client-server database?

Solo project, free-tier hardware, real money — I optimized for zero operational overhead. One writer, single file, no server to babysit, and WAL mode gives me concurrent reads. For the actual write load (one process, a handful of trades a minute) it's plenty. I'll be honest about the boundary: it bit me twice — mmap caused SIGBUS on the 1 GB box, and concurrency gets dicey if I ever had multiple writers. The day this becomes multi-process or multi-operator, I move to Postgres. But for what it is, SQLite was the right call, and I can tell you exactly where the line is.

3. Why eventlet / how does your concurrency model work?

Two layers. Eventlet green threads give me cooperative async I/O in a single process, which is what Flask-SocketIO wants for pushing live ticks and P&L to the browser. On top of that, each strategy runs in its own OS thread. Most of my hardest bugs lived exactly at that boundary — thread join semantics, heartbeat starvation, and eventlet's SSL monkey-patching breaking blocking HTTP libraries. It's not the model I'd pick at scale, but for one process serving a realtime UI it kept the operational footprint tiny.

4. How do ~31 strategies share market data without hammering the API?

The MarketDataService facade is the single choke point. When ten strategies want the same NIFTY candle, that's one fetch, cached, not ten. On top of caching there's a process-wide throttle (~3 req/s) so bursts can't stampede the vendor, and exponential backoff when we do get throttled. Before this I was doing ~1,400 API hits per host per day and getting rate-limited blind; after, about 90% fewer hits. The freshness of each cached value is tracked and exposed as a metric so "is this data stale?" is observable, not a guess.

5. What happens to open positions if the process restarts?

This was a real failure mode early on — a restart dropped in-memory state while real positions stayed open, so the strategy came back blind and re-entered. Now the **TradeResumer** restores open trades from SQLite on boot — it *resumes*, it doesn't re-fire entries. And the **Reconciler** continuously checks for drift between what the broker says I hold and what my state says, so if a restart ever did lose something, I'd see it as a drift metric

rather than discover it as an orphaned position later.

6. How do you detect and recover from a stuck strategy?

Every strategy emits a heartbeat from its work loop roughly every 30 seconds via a heartbeat-aware sleep. The **Watchdog** recycles any strategy whose heartbeat goes stale past a threshold. The subtlety is distinguishing "deliberately idle" from "genuinely hung" — an idle wait loop still heartbeats, so it's not killed, but a strategy wedged on a blocking call outside the sleep goes stale and gets recycled. Recovery is a two-phase recycle: stop, drain, then restart — never two live threads at once.

7. Walk me through your hardest production bug.

→ Tell story (1) (duplicate orders). Journal showed the same option bought 4× in a minute → traced to the watchdog → `stop()` flipped a flag but never *joined* the thread → if the strategy was hung, `start()` spawned a second thread → two threads each firing entries. Fix: spawn guard + two-phase recycle. Lesson: a kill that doesn't join is not a kill.

8. How do you handle a data feed going down or going stale?

Two cases. *Down* — the facade has multi-vendor fallback (Angel → Dhan → yfinance), so one vendor dying doesn't blind the system. *Stale* — this is the nastier one, and I learned it the hard way: NSE silently 403-blocked my cloud IPs, so the feed didn't error, it just stopped updating. Now freshness is a first-class tracked metric, and critical logic treats "data this old" as unusable rather than trusting it. The facade also meant migrating off NSE entirely was a one-module change, not 31.

9. How do you make sure backtests reflect live behavior?

The backtest engine replays the **real strategy code** — same class, same logic — against recorded data. The old version used per-strategy adapters (reimplementations) and they drifted; different code means the backtest is lying. I also added a data-fidelity grade so I know how trustworthy a given run's inputs were, and it runs as a niced subprocess so a heavy backtest never starves the live trader.

10. How do you avoid placing duplicate or erroneous orders?

Duplicates: the root fix was the thread spawn guard so a strategy can't run twice (story (1)), plus a re-entry guard that refuses a new position group while untracked legs are still held, plus a 15:25 EOD square-off net as a backstop. Erroneous: I never fabricate a price — if the live price is missing, I retry then skip, never fill at a made-up number (that fabrication once cost me a ₹33k phantom loss). In a money system a wrong number is worse than no number.

11. How do you run real-time + analytics workloads on a 1 GB VM?

Discipline forced by constraints. DuckDB defaults to ~80% of RAM, which over-committed a 1 GB box and caused SIGBUS via mmap'd Parquet — so I capped its memory and threads and gave it a disk-spill temp dir, all env-overridable so the 24 GB box can use more. Data is hot/cold tiered: recent stuff in SQLite, historical in Parquet read colum-

nar by DuckDB. Heavy work runs niced or offloaded so it can't starve the trader. systemd Restart=on-failure is the floor.

12. What's your observability setup and how do you actually use it to debug?

VictoriaMetrics for metrics, Grafana for dashboards, Loki for logs, GlitchTip for error tracking, process-exporter/Alloy for host metrics — across both VMs. The honest answer to "how do you use it" is that almost every bug I've described *started* at a dashboard or log, not in the code. Duplicate orders showed up in the trade journal, the recycle storm in watchdog logs, the feed outage as a freshness metric going flat. I also track per-strategy P&L, data-source freshness, and config changes correlated with P&L.

13. How do you deploy safely to two hosts without downtime surprises?

A one-command multi-host deploy script: it does a fast-forward-only pull, a compile-check, then restart, then a health-check, per host. CI runs ruff + pytest + gitleaks + pip-audit on every push. DB schema changes happen after market close, with a backup snapshot to OCI Object Storage first. I hardened the deploy to fail *loudly* after a silent no-op — because a deploy that "succeeds" without actually changing anything is its own bug.

14. How do you handle expired-option settlement?

Expiry is a *time* fact, so I derive it from the clock and the option symbol — never from a price feed, because expired options stop ticking. A state-independent sweep reads my actual positions, parses the expiry out of each symbol, and settles expired legs at intrinsic-vs-spot with no LTP needed. The paper engine accepts a ₹0 settlement *only* for a genuine expired close. This came out of a bug where an Iron Condor held expired legs for a day because the exit was gated behind a price that no longer existed.

15. Paper vs live — how do you keep them consistent?

They share the same code path up to the engine boundary — same OrderManager, same signals; only the final execution differs. The PaperEngine mirrors the live fill path as closely as possible. Live is opt-in per strategy, defaulting to paper, so going live is a deliberate switch, not the default risk.

16. What's your testing strategy for a system that touches money?

Unit tests on the money paths (entry/exit/settlement), parity tests where old implementations act as oracles for bit-for-bit comparison, and the backtest engine itself as a behavioral check since it runs the real code. My honest weakness: I shipped fast and back-filled tests *after* incidents — the phantom-fill and blank-DB bugs both ended with "...and I added the test that would've caught it." If I rebuilt it I'd front-load that, especially an end-to-end paper-trading smoke test in CI.

17. How do you handle secrets and security (broker keys, OAuth)?

Broker keys and tokens live in environment/config outside the repo, never committed — and gitleaks runs in CI to enforce that. Zerodha live trading uses OAuth, so there's a daily auth flow rather than long-lived credentials. pip-audit and Dependabot watch the

dependency surface. nginx terminates TLS in front of the app.

18. What were the biggest trade-offs you made and why?

Operational simplicity vs concurrency-safety (SQLite + single process — right for solo, wrong at scale). Data freshness vs API budget (caching with bounded TTL vs rate limits). Reliability vs latency (retry/skip instead of fabricating a fill). And feature velocity vs test coverage — I moved fast and paid it back after incidents, which I'd reverse next time. The theme: I consistently chose what one person could *operate*, and accepted the failure modes that came with it as long as I understood them.

19. What would you change if you rebuilt it today?

Postgres instead of SQLite — kills the mmap/concurrency bug class outright. Front-load tests, especially an automated end-to-end paper smoke test in CI so a regression like the blank-DB one can't ship. Reconsider eventlet for a cleaner async or a real task runner. And I'd build the reconciliation loop *first* rather than after the first orphaned position taught me I needed it.

20. What was the role of the watchdog/reconciler/resumer trio?

They're defense in depth for a long-running stateful process. **Watchdog** = liveness detection and recovery (recycle stuck strategies). **TradeResumer** = crash recovery (restore open trades on boot so a restart doesn't go blind). **Reconciler** = correctness (detect drift between my state and broker reality). Each one was added in response to a real incident, not built speculatively — detection, recovery, and reconciliation are the three things a stateful money system can't live without.

8 7. Top 10 deep-dive technical questions --- with answers

1. Why does an mmap page fault surface as SIGBUS, and why does it kill the whole process?

When you mmap a file, the kernel maps pages lazily — they're faulted in on first access. A normal SIGSEGV is "you touched memory you don't have rights to." SIGBUS is different: it's "the mapping is valid but the kernel *can't satisfy the backing*" — e.g. the page is beyond the file's actual length, or there's no physical memory/swap to back it under pressure. On the 1 GB box, DuckDB over-committing plus mmap'd Parquet meant a fault couldn't be backed, so the kernel delivered SIGBUS. And because it's a synchronous fault on a normal memory access — not a syscall returning an error you can check — the default disposition terminates the process. You can't `try/except` a bus error; the only real fixes are *don't over-commit* (memory caps) and *don't rely on mmap under pressure* (disk spill, disable SQLite mmap).

2. Heartbeat design: deliberately-idle loop vs genuine hang?

The key decision is *where* the heartbeat is emitted. If I beat from a background timer, every process that's merely *up* looks alive — useless. So the heartbeat is emitted from inside the work loop, via the sleep primitive itself: `self._sleep()` breaks a long wait into short

interruptible chunks and beats every ~30s. So a strategy that's *deliberately* idle (waiting for the next session) keeps beating — not killed. But a strategy stuck *outside* the sleep, on a blocking call, never reaches the next beat, goes stale, and gets recycled. The liveness signal has to be coupled to forward progress, not to the process existing.

3. eventlet monkey-patching failure modes, and how `patcher.original()` fixes them?

eventlet replaces stdlib socket/SSL/threading with cooperative versions so blocking calls yield to the green-thread scheduler instead of blocking the whole process. The leak: some libraries — certain SSL paths in Telegram/yfinance — don't tolerate the patched stack and recurse infinitely or deadlock, because they make assumptions the green version violates. `eventlet.patcher.original("urllib.request")` hands back the *real, un-patched* module, so those specific calls use genuine blocking sockets on a real OS thread while everything else stays cooperative. The lesson: global monkey-patching needs a per-call escape hatch, because you can't predict every library's assumptions.

4. SQLite WAL: what concurrency does it give, and where does it stop?

WAL (write-ahead logging) decouples readers from the writer: readers see a consistent snapshot and don't block the writer, and the writer appends to the WAL instead of locking the main DB file. So *many readers + one writer* run concurrently — perfect for my one-process trader with a realtime UI reading. Where it stops: it's still **one writer**. Two processes writing serialize hard (and can hit "database is locked"). WAL also needs periodic checkpointing or the WAL file grows. And PRAGMA `mmap_size` on top of WAL reintroduced the SIGBUS risk. So WAL bought me exactly what I needed (concurrent reads) and nothing it couldn't give (multi-writer) — which is the Postgres trigger.

5. Two-phase recycle: the states between stop and start, and the race you closed?

The original race: `stop()` set `_active = False` and returned immediately; `start()` checked `_active` and spawned. But the *thread* could still be alive (hung on a blocking call) when `start()` ran — `_active` being False doesn't mean the thread has exited. So you'd get thread A (dying, but not dead) and thread B (fresh) both live, both firing entries. Two-phase recycle splits it: phase one signals stop and waits for the thread to actually wind down (the drain); only phase two, on a later pass once the old thread is confirmed gone, does the restart. The guard is `start()` refusing to spawn while `_thread.is_alive()`. The closed window is the gap between "asked to stop" and "actually stopped."

6. How does the Reconciler define "drift," and avoid false positives on in-flight orders?

Drift = a broker position with no matching trade-state on my side (an orphan), or trade-state with no matching position. The false-positive risk is exactly the in-flight window — I've sent an order, the broker hasn't confirmed the position yet, so momentarily they disagree legitimately. The reconciler treats drift as a *metric* (`trader_position_drift`) to observe and alert on, with tolerance for transient states, rather than an auto-corrector that might cancel a real in-flight trade. You want it to *tell* you about drift loudly, not act blindly — auto-remediation on a money system is how you turn a glitch into a loss.

7. DuckDB over Parquet hot/cold tiering — what's hot, and the read path?

Hot = recent/live data the trader needs right now, kept in SQLite for low-latency point reads. Cold = historical bars for backtesting/analytics, written to Parquet files partitioned by date/symbol. The read path for analytics: DuckDB queries the Parquet directly — no import step — using columnar vectorized scans with projection and predicate pushdown, so a query for one symbol's close over a date range only touches the relevant column chunks and row groups, not the whole file. The cutover policy is age-based: once data ages out of the hot window it's archived to Parquet.

8. Backtest fidelity grading — what determines the grade, and why it matters?

The grade reflects how trustworthy the *input data* was for a given run — gaps in the recorded ticks, missing candles, low-liquidity symbols where prices are unreliable, resolution mismatches between what the strategy needs and what was recorded. It matters because a backtest's headline number is meaningless if the data feeding it was full of holes — without the grade you'd treat a run on pristine data and a run on swiss-cheese data as equally credible and over-fit to noise. The grade turns "this backtest says +20%" into "this backtest says +20% *and here's how much you should believe it.*"

9. Rate-limit triad — sizing cache TTL vs throttle vs backoff so they don't fight?

They operate at different layers, so they compose rather than conflict. **Cache TTL** is sized to the data's natural update rate — a 1-minute candle doesn't need re-fetching within the minute, so $TTL \approx$ the bar period; that alone collapses N strategies into one fetch. **Throttle** (~3 req/s) is the steady-state ceiling that protects the vendor from bursts the cache *doesn't* absorb (lots of distinct symbols). **Backoff** (5→10→20→40s) only engages on an actual throttle response — it's the reactive safety valve when the first two underestimate. The trick is the cache absorbs *redundant* load, the throttle shapes *distinct* load, and backoff handles *being wrong* — each targets a different cause, so tuning one doesn't destabilize the others.

10. Settling an expired option without a price — intrinsic-vs-spot, and edge cases?

Cash-settled index options settle at intrinsic value on expiry: for a call, $\max(0, \text{spot} - \text{strike})$; for a put, $\max(0, \text{strike} - \text{spot})$. So I don't need the option's last price at all — I need the *underlying* spot and the strike (which I parse from the symbol). I close shorts as BUY and longs as SELL at that intrinsic value. Edge cases: getting the right settlement spot (the exchange uses an average, I approximate with the underlying at expiry); deep OTM legs settle at ₹0, which is exactly why the paper engine had to *allow* a ₹0 fill for genuine expired closes; and MCX vs NFO symbols parse differently, so the symbol parser has to handle both formats.

9 8. Top 10 follow-up questions (experienced interviewer) --- with answers

1. "Why not just add a join instead of a two-phase recycle?"

Because a blocking `join()` in the watchdog would freeze the watchdog itself on the exact thing it's trying to fix — a hung thread. If the strategy is wedged on a network call that

never returns, `join()` blocks forever (or I'd need a timeout, and then I'm back to "is it really dead?"). Two-phase decouples it: signal stop, return immediately, and on a *later* watchdog pass check `is_alive()` before restarting. The watchdog stays responsive and never spawns a duplicate. A join couples the supervisor's liveness to the worker's death — the opposite of what you want.

2. "Heartbeat every 30s but timeout 600s — how'd you pick those, and the latency cost?"

30s is well inside the timeout so a healthy-but-busy strategy has many chances to beat before it's ever suspected — avoids false kills from one slow cycle. 600s (vs the original 300s) came directly from an incident: strategies with a poll interval ≥ 300 s raced the 300s watchdog and got recycled mid-work. Widening to 600s gives genuine slow strategies headroom. The cost is detection latency — a truly hung strategy can sit dead for up to ~10 minutes before recycle. For my strategies, which mostly act on minute-or-slower signals, 10 minutes of one strategy being down is acceptable; false-killing a healthy one mid-trade is worse. It's a deliberate latency-vs-false-positive trade.

3. "If you'd had Postgres from day one, which bugs disappear and which remain?"

Disappear: the SIGBUS-from-SQLite-mmap, the concurrency/locking risk, and arguably the path-depth blank-DB bug would've been *louder* (a connection failure vs a silently empty file). Remain: everything that wasn't about storage — the thread-duplicate race, watchdog heartbeat starvation, the expired-option strand, the fabricated fill, the event-let SSL recursion, the NSE feed outage. That's the honest answer — Postgres fixes a *class* of bug (storage/concurrency), but the majority of my hard bugs were concurrency and external-dependency problems that no database choice touches.

4. "Caching market data sounds dangerous — how do you bound staleness?"

The cache is for *fetch deduplication*, not for serving arbitrarily old prices. TTL is tied to the data's update cadence — a minute candle is valid for its minute. More importantly, freshness is tracked and exposed as a metric, so staleness is *observable*, and the genuinely dangerous path — critical exits/stop-losses — doesn't act on a stale or missing price at all; it falls back to intrinsic-vs-spot. So the cache speeds up reads without ever being the thing that decides a trade on bad data. The NSE incident actually taught me this: staleness is the silent killer, so I made it loud.

5. "How do you *know* you removed every other fabricated value?"

I can't claim 100%, and I'd say so. What I did: grepped out every `or <number> fall-back` in the price/fill paths, removed them, and replaced fabrication with explicit `retry→skip→intrinsic`. Then I added tests that assert a missing price results in *no trade* rather than a default fill — so a regression reintroducing fabrication would fail CI. The honest framing is: I eliminated the ones I found and built a guardrail so new ones get caught, but "prove a negative across the whole codebase" is exactly why I'd want broader money-path test coverage if I rebuilt it.

6. "Backtest runs real code — but recorded data isn't live. Where can it still lie?"

Several places, which is why the fidelity grade exists. Recorded data has no real fill modeling — I assume I get filled at the recorded price, but live I'd face slippage, partial fills, and liquidity limits, especially on the illiquid option strikes my strategies trade. There's no latency in replay — live, the gap between signal and order matters. And survivorship/-gap issues in the recorded set. So the backtest is faithful to my *logic* but optimistic about *execution* — it tells me if the strategy reasons correctly, not that I'd capture the same P&L live.

7. "Why didn't an idempotency key at the order layer save you from the duplicate orders?"

Because the two threads weren't retrying the *same* logical order — they were two independent strategy instances each *independently deciding* to enter, generating distinct order intents at slightly different times/prices. An idempotency key dedupes "the same order sent twice," but this was "two correct-looking-but-distinct orders from a process that should only exist once." The bug was upstream of the order layer — at process lifecycle. An idempotency key would've been a band-aid over a duplicated *actor*; the real fix had to prevent the second actor from existing.

8. "How do you ensure a systemd restart loop doesn't itself cause duplicate trades?"

The restart path goes through the *same* recovery as any boot: TradeResumer restores open trades and *resumes* monitoring rather than re-firing entries, and the re-entry guard refuses a new position group while legs are already held. So even if systemd restarts repeatedly, each boot reconciles against persisted state instead of opening fresh positions. The Reconciler would also surface any drift a crash-loop introduced. Restart safety = idempotent boot, not just "start the process again."

9. "Adding safety only after incidents — isn't that just reactive engineering?"

Partly, and deliberately. For a solo free-tier system, building speculative safety for failures that may never happen is its own waste — over-engineering. What incidents gave me was the *real* failure mode, so the safety I built is targeted and proven, not cargo-culted. What I'd defend as the mature part isn't "I reacted" — it's that each reaction ended with a *systemic* fix and a regression test, so the same class can't recur. Proactive would've looked like: reconciliation and end-to-end paper smoke tests from day one, since those are generic to any stateful money system — and that's exactly what I'd front-load if rebuilding.

10. "What's your blast radius if the broker returns *wrong* data, not no data?"

That's the scariest case and I'll be straight: wrong-but-plausible data is the hardest to defend against, because my whole "never fabricate, treat missing as missing" design handles *absence*, not *lies*. Current mitigations are partial — sanity bounds on prices, cross-checking the underlying spot against the option-derived value, and the Reconciler catching position-level disagreements. But a subtly wrong LTP that passes bounds could drive a bad exit. If I were hardening this, I'd add cross-vendor agreement checks (does Angel's price match Dhan's within a tolerance?) and circuit-breaker logic on implausible moves. It's a known gap, not a solved problem.

10 9. Where the interviewer may challenge you --- and how to defend

Challenge	Defense
"SQLite in a trading system is amateur."	One writer, free-tier, zero-ops was the right call for solo scale; I name the exact failure points (mmap SIGBUS, concurrency) and the migration trigger to Postgres. I understood the boundary, didn't stumble into it.
"Eventlet monkey-patching is fragile."	Agreed, and I hit it (SSL recursion). It bought single-process Socket.IO + async I/O with minimal ops. I have the escape hatch (<code>patcher.original()</code>) and know when I'd move to a real task model.
"Caching market data risks acting on stale prices."	Cache is for <i>fetch dedup</i> across strategies with bounded TTL and freshness tracking exposed as a metric; critical exits never act on a price that's gone — they fall back to intrinsic-vs-spot.
"You only added safety after incidents — that's reactive."	True and deliberate: I didn't over-engineer speculative safety for a solo system. Each incident taught the <i>real</i> failure mode, so the safety is targeted, not cargo-culted. The meta-fix was adding the regression test each time.
"A 15:25 EOD square-off net is a band-aid over the duplicate-order bug."	The spawn guard is the real fix; the EOD net is defense-in-depth, not the primary control. In a money system you want both the fix <i>and</i> a backstop.
"Backtest-on-real-code still trusts your recorded data."	Hence the data-fidelity grade — it tells you how much to trust a given run rather than pretending all backtests are equal.

11 10. Things to study/refresh before the interview

- **Memory & mmap internals:** why a fault on an mmap'd page → SIGBUS vs SIGSEGV; OOM-killer vs over-commit; swap behavior under pressure.
- **SQLite WAL semantics:** readers vs the single writer, checkpointing, where WAL stops scaling, and PRAGMA `mmap_size` risks.
- **eventlet / green threads:** cooperative vs preemptive scheduling, what monkey-patching replaces, and which libraries break.
- **Thread lifecycle in Python:** join, daemon threads, the GIL's role here, why "set a flag" isn't a stop.
- **Rate-limit patterns:** token bucket vs leaky bucket, jittered exponential backoff, idempotency keys.
- **Options settlement math:** intrinsic value, expiry mechanics (cash-settled index options), why premium → 0 at expiry.
- **Observability vocabulary:** RED vs USE methods, what makes a good SLI, cardinality pitfalls in metrics.
- **DuckDB execution model:** vectorized columnar scans, predicate/projection pushdown into Parquet, memory limits.
- **Crash-consistency / reconciliation:** idempotency, exactly-once vs at-least-once, the "ledger vs reality" reconciliation pattern.
- **Be ready to whiteboard:** the watchdog/heartbeat state machine and the two-phase recycle, since those are your strongest stories.

12 11. One-line closer

"It's a solo project, but I treated it like production — because it trades real money. Most of the engineering went into the boring-but-hard parts: not crashing, not double-trading, not trusting a single feed, and always being able to *see* what it's doing."